

Verity: How It Works and Verification Report

Written Saturday, September 5, 2026, ~1:30 PM ET, from the live state of ~/Documents/Projects/verity, the running Next.js dev server, the Supabase project, and the DuckDB warehouse. Everything in this document was checked by running it, not by reading it.

1. What the product is, in one paragraph

Verity is a pre-payment program-integrity console for Medicaid and Medicare. It continuously scores every enrolled provider in the country against fourteen public federal datasets, surfaces three kinds of fraud that today are found months later by retrospective audit (ghost provider networks, impossible billing days, and providers still being paid after a federal or state action), and turns each finding into a referral packet that a human program-integrity analyst reviews and signs. The reviewer's decision feeds back into the scoring. Nothing in it touches a beneficiary, denies a claim, or contains PHI.

2. The user's experience, page by page

The user is a program-integrity analyst at a state Medicaid agency, a CMS Center for Program Integrity reviewer, or a Task Force to Eliminate Fraud staffer. The interface is a custom Next.js 15 app (Tailwind 4, serif display headings, a light neutral palette, rounded cards, D3 county map and force-directed ownership graph). It is not USWDS.

Landing page (/)

A product site: "Real-time detection and incident response," a rendered mock of the console, sections on how continuous scoring works, and an "Open console" button. Purpose: orient a judge or buyer in ten seconds.

Overview, the "National tripwire" (/app)

Four KPI tiles across the top, each a link: Medicaid dollars paid after a federal or state action (with the NPI count), rendering NPIs with an impossible month (with dollars paid in those months), provider communities ranked (with Medicaid 2024 dollars in the top 200), and rows scanned (with NPPES and Medicaid provider counts). Under them, a zoomable county choropleth of dollars at risk on a log scale; hovering a county shows its at-risk total, how many communities and flagged NPIs it contains, the split across the three detectors, and the top community ID. Below the map, two panels: "Top communities" (rank, city, score, one-line summary, provider count, dollars) and "Top flags" (provider name, detector tag, metric, value). Every element links deeper. A "show the math" link goes to Methods.

Clusters (/app/clusters) and a cluster (/app/clusters/{id})

The ranked list of provider communities from Detector 1, filterable by state via pills; chains and private-equity platforms are scored but held out unless the user clicks "include chains." Columns: rank, community ID, where, providers, score, Medicaid 2024, Medicare 2023, and a "why" summary. Opening a community shows the ownership graph (entities, owners, addresses, phones as nodes; the excluded or revoked ones highlighted), the member table sorted by Medicaid 2024 dollars, the owner rows, any revocations or LEIE entries among members, the county's Market Saturation context (providers per 10K FFS beneficiaries, moratorium flag), and the Referral packet panel.

Flags (/app/flags)

Two tabs. "Revoked but paid" (Detector 3): one row per NPI with the source list (Medicare revocation, OIG LEIE, SAM.gov, state exclusion, NPES deactivation, Medicaid deceased), the action and its date, months paid after, dollars paid after, tier (A = integrity grounds, B = administrative) and score. "Impossible days" (Detector 2): the peak month, the label, hours per day under the conservative test, the lines-based lower bound (which needs no rate assumption at all), how many billing organizations submitted for that rendering NPI, and dollars paid that month. Tier A is the default; "all tiers" widens it. Each row links to the provider.

Provider (/app/providers/{npi})

The provider's NPES identity (name, entity type, taxonomy, Medicaid home state), a "Federal actions" card listing every revocation, exclusion, debarment, state action or deactivation with dates and the regulatory ground, the Medicaid payment timeline with the action date marked so "paid after" is visible, any impossible-day months, and the Referral packet panel.

The Referral packet panel (on every cluster and provider page)

One button: "Generate packet." The investigator agent reads the evidence rows for that subject and nothing else, drafts a CMS-style referral with a title, summary, numbered findings, the 42 CFR grounds that apply (424.535(a)(2), (a)(8), (a)(12), (a)(19), 455.23, 455.416(c), 1001.1901, with plain-English descriptions), a recommendation, and a plain-English explanation. Every finding cites the evidence row ID it came from; sentences without a citation are dropped. When no OpenAI key is present the deterministic builder produces the same packet from the evidence lines, so the demo never depends on a model. The packet is saved to Supabase and to a private storage bucket. The reviewer then clicks Accept, Reject, or Needs info, optionally with a note. The decision is stored, the packet status changes, and the detector's evidence-family weights are re-estimated from all reviews so far (Bayesian re-weighting with a prior of four pseudo-reviews); the new weights are shown on screen. That is the feedback loop the OpenAI and CMS judges ask for.

Minnesota (/app/states/MN) and Methods (/app/methods)

The state page reproduces, transparently, what CMS did opaquely: Minnesota's flagged providers, the impossible-day cases with the published DHS fee schedule used to convert dollars to hours, and the dollars involved, with the math shown. Methods renders docs/methods.md, which the detector code itself writes: every threshold, every dropped row, every rate estimator and its validation against Minnesota's published rates, and every table's row count. "Every number, show the math" is a product feature because the states are demanding exactly that from CMS.

API and integrations the user does not see

A FastAPI service (api/main.py) exposes the same evidence, packet and review operations, plus the NPI tripwire (/verify/npi/{npi}, POST /verify for a list of claims), a live SAM.gov exclusions lookup, and the Blue Button 2.0 flow for a Medicare beneficiary to run their own claims history through the tripwire. Blue Button is wired to the production API and requires CMS production approval; the sandbox is blocked because it only serves synthetic beneficiaries.

3. How the pipeline works, end to end

1. **Download** (`scripts/download_all.py`): 268 files, 135 GB, from `data.cms.gov`, `opendata.hhs.gov`, `OIG`, `NPPES`, `SAM.gov`, `Open Payments`, `openFDA` and `Care Compare`, each with its source URL recorded.
2. **Clean and load** (`ingest/00` to `06`): re-encode the CMS CSVs, build the DuckDB warehouse (54 tables, 19 GB); the 56.7M-row time-coded Medicaid spend table, 49.5M enrollment segments, 613K owner rows, 9.7M NPIs, the exclusion sources (revoked, LEIE, SAM, state lists, deactivations), Market Saturation, the 320-code time table, Minnesota published rates.
3. **Detect** (`detectors/d1`, `d2`, `d3`): each writes its output tables and appends its own section to `docs/methods.md`. D3 is done (481,203 events, 1,106 paid-after cases, 962 flags). D2 is finishing (29.6M scored NPI-months). D1 has not run yet.
4. **Serve** (`ingest/sync_outputs.py`): applies the Supabase schema and pushes clusters, cluster members, flags, providers, county risk rollups, the national summary and the methods text into Postgres. The web app reads Postgres through the public key with row-level security; packets and reviews are written server-side with the secret key.
5. **Review**: analysts work in the console; every packet and decision is persisted; weights update.

4. Verification results

Run with `python3 scripts/verify_all.py` (new; re-run it any time, it exits non-zero on failure). 43 passed, 9 pending, 1 failed.

Result	Check	Detail
PASS	All 268 manifest files downloaded, key files present	Medicaid parquet 3.07 GB, enrollment segments, LEIE, NPPES 1.15 GB, SAM extract; no synthetic Blue Button data on disk
PASS	Warehouse tables at expected scale	spend 56,701,897; enroll 49,470,292; owners 613,574; revoked 8,136; leie 83,842; sam 168,304 (19,215 with NPI); nppes 9,726,865; saturation_county 1,030,290; hospice 5,802; hha 11,494; snf 14,410; timecodes 320; state_exclusions 46,170
PASS	Detector 3 output	d3_paid_after 1,106 rows; flags 962 rows
PASS	Detector 2 output	d2_scored 29,562,858 rows (still being written at check time)
PENDING	Detector 1 output	clusters table not built yet; run <code>detectors/d1_ghost_networks.py</code>
PASS	Tripwire logic	Known NPIs return real events (MEDICARE_REVOKED, OIG_LEIE, SAM_HHS, STATE_EXCL_CA); a service date inside the revocation window is flagged; one before it is not
FAIL	SAM.gov live lookup during this run	HTTP 429 "message throttled." Non-federal SAM keys are rate-limited and my test calls hit the limit. It worked minutes earlier. Not a code defect; avoid hammering it in the demo, the bulk extract is already loaded.

PASS	Supabase reference tables	revoked 8,136; leie 83,842; owners 613,574; saturation_county 204,159; datasets 62; packets and reviews tables exist and are empty
PENDING	Supabase serving tables	clusters, cluster_members, flags, providers are empty; county_risk, summary, score_weights do not exist yet. All are created and filled by <code>ingest/sync_outputs.py</code> , which has not been run since the detectors started. This is why the console shows \$0 and 0 everywhere and why provider pages show no name.
PENDING	FastAPI service	Not running on port 8000. Start with <code>.venv/bin/uvicorn api.main:app --port 8000</code> . The module imports cleanly and the verify routes work when started.
PASS	Web app routes	<code>/</code> , <code>/app</code> , <code>/app/clusters</code> , <code>/app/flags</code> , <code>/app/flags?detector=D2</code> , <code>/app/states/MN</code> , <code>/app/methods</code> , <code>/app/providers/1811937436</code> all return 200 with no runtime error; TypeScript typecheck passes with zero errors
PENDING	Console shows non-zero KPIs	Waits on <code>sync_outputs</code>
PASS	Blue Button	Base URL is production; CMS token endpoint reachable. It returns 401 <code>invalid_client</code> in production, as expected, because the registered credentials are sandbox credentials. Production credentials come only from CMS approval.

Defects found and fixed during the check

- **County map rendered as a thumbnail in the corner of its card.** Cause: the installed `us-atlas` counties file is in longitude and latitude (bounding box -179 to 180), but the map drew it with an identity projection meant for a pre-projected atlas. Fixed in `web/components/app/CountyMap.tsx`: detect degree coordinates and project with Albers USA fitted to the card. Typecheck passes. The running dev server did not pick the change up through the file mount, so restart it (Ctrl-C, `npm run dev`) and reload `/app`; the map should fill the card.
- **Provider page label read "under 1128 1128b4."** Fixed to "under section 1128b4."
- **Blue Button scopes in `.env` were invalid** (fixed earlier today) and **the SAM key line had been corrupted by an append** (fixed earlier today).

What must happen, in order, before the console is demo-ready

1. Let D2 finish (`logs/d2.log` ends with its summary).
2. Run `.venv/bin/python detectors/d1_ghost_networks.py` (writes clusters and cluster_members to the warehouse and its Methods section).
3. Run `.venv/bin/python ingest/sync_outputs.py` (creates `county_risk`, `summary`, `score_weights`; fills clusters, flags, providers; publishes `methods_md`).
4. Restart `npm run dev`, open `/app`, confirm the four KPIs are non-zero, the map is filled, and a top community opens with its graph.
5. Start the API (`.venv/bin/uvicorn api.main:app --port 8000`) if the packet agent path through FastAPI is used; the web app's own `/api/packets` and `/api/reviews` routes work without it.
6. Generate one packet on the demo cluster and one on the demo provider, accept one, and confirm the weights panel updates. Then run `python3 scripts/verify_all.py` again; everything should read PASS.

7. Commit. The repo still has no commits.

5. The demo path, as the audience sees it

Open /app. Read the four numbers out loud. Zoom the map into Los Angeles County, hover, click the top community. On the cluster page, point at the graph: several hospices incorporated within weeks of each other, sharing an owner and an address, one owner on the OIG exclusion list, in a county the Market Saturation file already marks as oversupplied. Click Generate packet; read the first finding and its citation; click Accept; show the weights move. Go to Flags, Impossible days, open the Minnesota provider with the highest hours per day; show the lower-bound column (no rate assumption) and the published rate used for the point estimate. Go to Flags, Revoked but paid; open a provider whose Medicare revocation date sits before months of Medicaid payment; generate the packet. End on Methods: "every number on the previous screens is computed by the code that wrote this page." Three numbers to close: Medicaid dollars paid after a federal or state action, NPIs with an impossible month, and minutes per packet versus days per retrospective audit.